

Brac University
Department of Electrical & Electronic Engineering
Fall2025



Inspiring Excellence

Course Number : EEE282

Section : 01

Group No : 04

Lab Report

Experiment no: 06

Name of the experiment:

Curve Fitting: Linear & Polynomial Interpolation

Prepared by:

Name: Sabrina Maliha ID: 23221082

All Group members:

<i>Sl.</i>	<i>ID</i>	<i>Name</i>
1.	24121205	Souvik Barman Ratul
2.	24121308	Alif Tamjid
3.	23221082	Sabrina Maliha
4.	24121019	Monazzi Siddique

(1)

Objective:

The objective of this experiment is to apply curve fitting techniques using linear and polynomial interpolation to analyze discrete sets of data. It aims to estimate unknown values between measured points, construct accurate interpolating functions, and compare the effectiveness of different fitting methods in minimizing deviation between actual and estimated results.

Theoretical Background:

Polynomial interpolation is a mathematical technique that helps to determine a unique polynomial function that passes exactly through a set of given points. It is used to determine the values between known data points with the assumption that the values come from a smooth function. For $n+1$ data points, a unique polynomial can be made that passes through each of the data points exactly.

Among the two common types of interpolation (Linear Interpolation and Polynomial interpolation), the polynomial interpolation is appropriate for the given problem as the provided data points come from a high precision. Polynomial interpolation can give value exactly at the given 3.5 value. The Lagrange interpolating polynomial is the polynomial of $P(x)$ of degree $n-1$ that passes through the n points. The formula to calculate Lagrange interpolating function is

$$P(x) = \frac{(x-x_2)(x-x_3)\dots(x-x_n)}{(x_1-x_2)(x_1-x_3)\dots(x_1-x_n)} y_1 + \frac{(x-x_1)(x-x_3)\dots(x-x_n)}{(x_2-x_1)(x_2-x_3)\dots(x_2-x_n)} y_2 + \dots + \frac{(x-x_1)(x-x_2)\dots(x-x_{n-1})}{(x_n-x_1)(x_n-x_2)\dots(x_n-x_{n-1})} y_n.$$

That is

$$P(x) = \sum_{j=1}^n P_j(x),$$

Where

$$P_j(x) = y_j \prod_{\substack{k=1 \\ k \neq j}}^n \frac{x - x_k}{x_j - x_k}.$$

Interpolation serves as an important tool to discrete experimental data points as well as enables estimation within measured ranges and constructs smooth interpolating functions.

Problem Statement:

Problem-1: The following data come from a table that was measured with high precision. Use the best numerical method (for this type of problem) to determine y at x = 3.5 using MATLAB. Note that a polynomial will yield an exact value.

x	0	1.8	5	6	8.2	9.2	12
y	26	16.415	5.375	3.5	2.015	2.54	8

Answer:

Code:

```
clear all;
```

```
close all;
```

```
clc;
```

```
x = [0 1.8 5 6 8.2 9.2 12]; % forming array of the given values of x.
```

```
y = [26 16.415 5.375 3.5 2.015 2.54 8]; % forming array of the given value of y.
```

```
n = length(x); % Number of element in array of x.
```

```
figure(1)
```

```
plot(x,y,'r','MarkerSize', 20); % Plotting the points in the graph.
```

```
title("Best numerical method 'Lagrange Interpolating Polynomial");% Giving title of the graph.
```

```
xlabel(' x ');% Labeling the x axis of graph.
```

```
ylabel(' y ');% Labeling the y axis of graph.
```

```
grid on;% to get grid lines.
```

```
hold on;%to keep the plot unchanged.
```

```
x_s = 0:0.01:12;%Continuous value of x from 0 to 12 with interval of 0.01.
```

```
x_s_n = length(x_s);%Number of elements in x_s.
```

```
y_s = zeros(1,x_s_n);%array equal to the number of elements in x_s with zero values.
```

```
for L = 1:x_s_n % For every value of x_s
```

```
    p = ones(1,n); % Array from 1 to length of x to store every value of P.
```

```
    for i = 1:n %For every point of x.
```

```
        for j = 1:n %For every point of x.
```

```
            if j~=i %To exclude the point when j=i.
```

```
                p(i) = p(i)* (x_s(L)- x(j))/( x(i)-x(j));
```

```
            end
```

```
        end
```

```
        p(i) = p(i)* y(i);% Each term of y_s.
```

```
    end
```

```
    y_s(L) = sum(p);% Sum of all y_s points.
```

```
end
```

```
figure(1)
```

```
plot (x_s,y_s,'r-');% Plotting graph.
```

```
grid on;% to get grid lines.
```

```
hold on;%to keep the plot unchanged.
```

```
tolerance = 0.001;
```

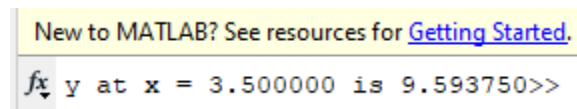
```
x_find = find((abs(x_s-3.5)<= tolerance));%Using find function to get in which array x_s=3.5 exist.
```

```
x_value = x_s(x_find);%x_s value stored from the array which contains x_s = 3.5.
```

```
y_s_find = y_s(x_find);%At that specific array corresponding value of y_s.
```

```
fprintf("y at x = %f is %f", x_value, y_s_find);% Showing the output value.
```

Output:



New to MATLAB? See resources for [Getting Started](#).

```
f_x y at x = 3.500000 is 9.593750>>
```

Figure 1(i): Value of y at $x = 3.5$ using best numerical method(Lagrange Interpolating Polynomial.)

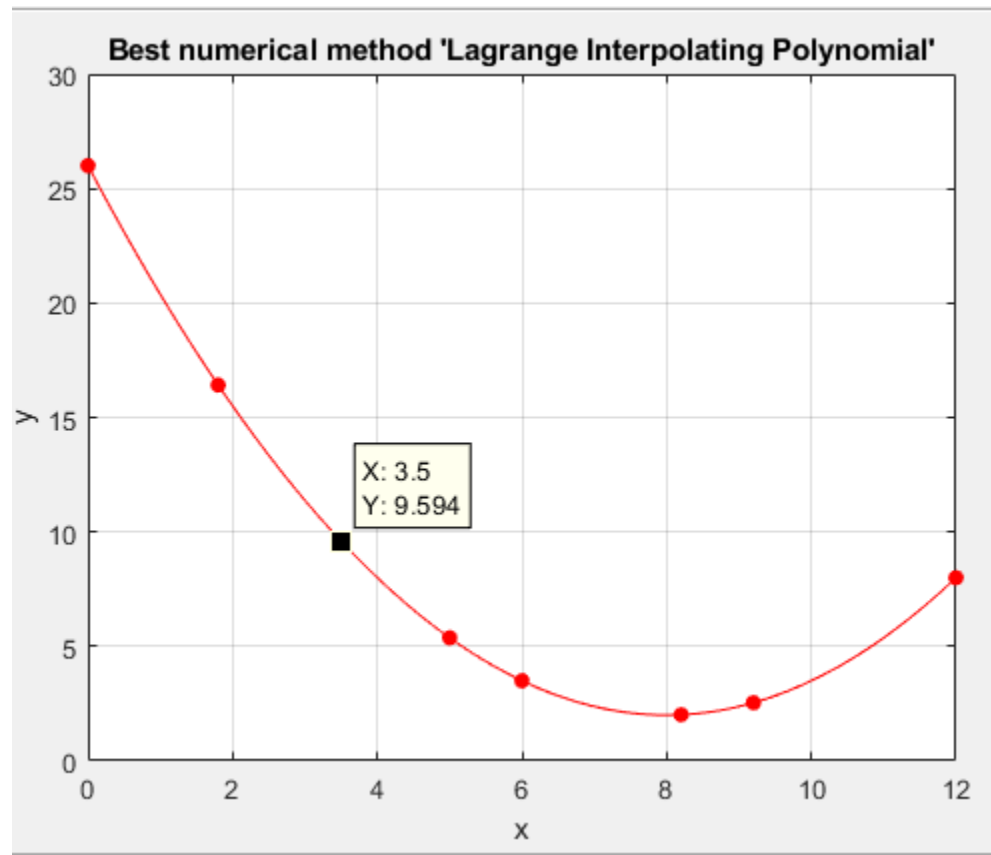


Figure 1(ii): Curve made using Lagrange Interpolating Polynomial.

Explanation:

Arrays (x and y) with the given values are made and their length is measured and the points are plotted in the graph. Then a series of values of x is taken and an array is made to store the corresponding interpolated values of y with the number of elements in x. The for loop is run for every value of the series of x. The condition j not equal to i is given to avoid repeating of values. Each term of the polynomial is constructed by multiplying factors like $\frac{(x-x_j)}{(x_i-x_j)}$ where $j \neq i$. The values get saved in the array P. The loop runs for every given data point of a series of values of x. Finally sums up all the values and plots it in the graph. To find the point 3.5 “find” command is used. If the difference of this point and the nearest point to this point is less or equal to 0.001, that point is the value of x whose corresponding value is the required output and is shown using the “fprintf” function. The variable x_value is used to get the value of x=3.5 from the array.

Discussion:

The given experiment approximates the value of y at $x = 3.5$ using the best numerical method. Among all the numerical approaches, Lagrange Interpolating polynomial is used as it goes exactly through all the points.

The given data were first stored in arrays in MATLAB and plotted to get a graph. A set of values of x from 0 to 12 was taken to get a smooth polynomial. Lagrange basis polynomial was found out by using the formula of products of $\frac{x-x_j}{x_i-x_j}$ for every value in this set. The summation was done after multiplying each of the basis polynomials by the corresponding values of y.

The resulting curve passes through all the given data points. Using find function and tolerance variable, the closest value to $x=3.5$ is found out. The MATLAB showed the corresponding values of x that is the output, y(3.5). Overall, the experiment shows the effectiveness of Lagrange Interpolating Polynomials to generate a smooth curve.

(2)

Objective :

The aim of this experiment is to approximate dissolved oxygen concentration at 27 degree Celsius using Linear Interpolation and Lagrange Interpolating Polynomials in MATLAB and to find the percentage error against the exact value.

Theoretical Background:

Interpolation is the technique to find unknown data points from the known data points. In this experiment, we have experimental results at some discrete temperature, interpolation helps us to determine the approximate dissolved oxygen concentration at any intermediate temperature. Commonly two types of interpolation are used.

Linear interpolation:

Linear interpolation approximates the function changes linearly between two data points. For instance if we know $y=4$ at $x=1$ and $y=8$ at $x=3$, we can estimate that at $x=2$, y is likely to be 6 using linear interpolation. Linear interpolation use two data points like for a point lying between x and x_0 , the approximate value of y is

$$y = y_0 + \frac{y_1 - y_0}{x_1 - x_0} (x - x_0)$$

Lagrange Interpolating Polynomial:

A Lagrange Interpolating Polynomial is a unique polynomial that goes exactly through a given set of data points, often approximating the function when the function is not known. For $n+1$ data points (x_i, y_i) the interpolating polynomial is

$$P(x) = \sum_{i=0}^n y_i L_i(x)$$

Where

$$L_i(x) = \prod_{\substack{j=0 \\ j \neq i}}^n \frac{x - x_j}{x_i - x_j}$$

This method gives exact fit for the given data and is more efficient than Linear Interpolation.

Percentage Error:

Percentage error is the difference between observed value and actual value and expressed as a percentage of actual value. That is

$$\%Error = \left| \frac{Exact\ value - Computed\ value}{Exact\ value} \right| \times 100\%$$

This method helps to analyze which interpolation method is better for environmental and scientific data.

Problem Statement:

Problem-2: The following data define the sea-level concentration of dissolved Oxygen for fresh water as a function of temperature:

T (°C)	0	8	16	24	32	40
Oxygen(mg/L)	14.621	11.843	9.870	8.418	7.305	6.413

Estimate Oxygen concentration using MATLAB for T = 27°C, considering:

- (a) linear interpolation,
- (b) Lagrange's interpolating polynomial.
- (c) Note that the exact result is 7.986 mg/L. Calculate the percentage error for both of the cases.

Answer:

(a)

Code:

```
clear all;
close all;
clc;
Temp = [0 8 16 24 32 40]; % forming array of the given Temperatures.
```

```

Ox_Con= [14.621 11.843 9.870 8.418 7.305 6.413]; % forming array of the given
concentrations of Oxygen(mg/L).
n = length(Temp); %Number of element in array of Temperature.
figure(1)
plot(Temp,Ox_Con,'r.','MarkerSize', 20);%Plotting the points in the graph.
grid on;
hold on;
Temp_s = 0:0.01:40; % Series is formed.
L = length(Temp_s); %Number of element in the series of Temp_s.
ys = zeros(1,L);%array of 0 elements are taken.
for i = 1:L % Loop works for every value of Temp_s.
    for j = 1: (n-1) %check between which point.
        if Temp_s(i)>= Temp(j) && Temp_s(i)<Temp(j+1)
            a =j; %lower index.
            b = j+1;%upper index.
        end
    end
    ys(i) =
Ox_Con(a)+((Ox_Con(b)-Ox_Con(a))./(Temp(b)-Temp(a))).*(Temp_s(i)-Temp(a));%Line
ar Interpolation formula.
end
figure (1)
plot (Temp_s,ys,'b.');//All the points are joined according to the Linear
Interpolation formula.
title('Plot using Linear Interpolation');
xlabel('Temperature (Celsius) ');
ylabel('Oxygen Concentration (mg/L)');
x_f = find(Temp_s == 27);% Locate the element.
y_f= ys(x_f);% Find value of oxygen concentration for which temperature is
27degree Celsius.
fprintf('\nOxygen concentration at T = 27 degree Celsius is %f',y_f);

```

Output:

```
Command Window
New to MATLAB? See resources for Getting Started.
Oxygen concentration at T = 27 degree Celsius is 8.000625>>
```

Figure 2(a)(i): Showing concentration oxygen at 27 degree Celsius using Linear Interpolation.

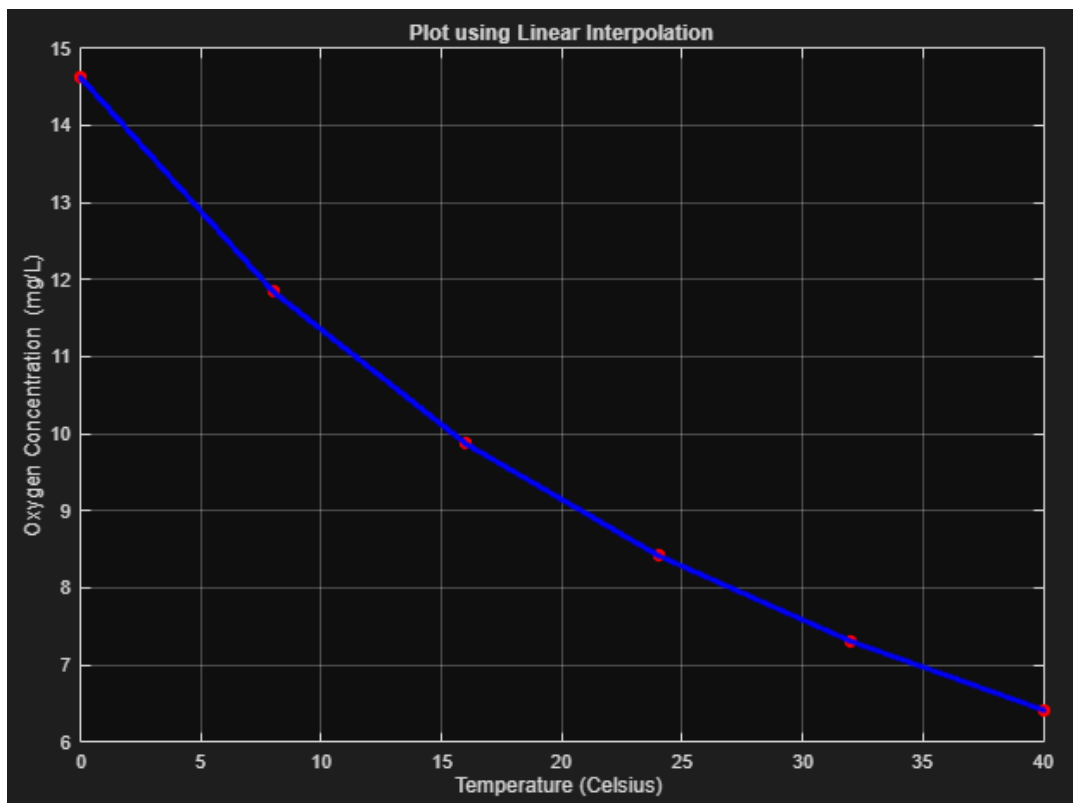


Figure 2(a)(ii): Graph showing points connected using Linear Interpolation Method.

Explanation:

The array is formed with the given data and plotted on the graph using 'plot' command with Temperature along the x axis and Oxygen concentration along the y axis. The point is plotted in red colored as "r" written inside the "plot" command. "Grid" command is used to get grid lines and the "hold on" command is used to keep the data points intact even after the next plot. For drawing a continuous line, the series of points for temperature is taken from 0 to 40 and its number of elements is measured using the "length" command. Then an array is formed for storing the corresponding values of x that are linearly interpolated y. A loop is run for every value of the series of x. And every time point is checked to know- it exists between which point. After confirming the position of the point, the formula of linear interpolation is used. In the graph these points are plotted. Using "title", "xlabel" and "ylabel", the title and the labeling of x and y axis are done respectively. Using find command temperature 27 degree is found out that is which array element it is. Then the corresponding value of temperature, that is level of oxygen concentration is found out by expressing its value as a function of oxygen concentration level. Finally the output is shown using the "fprintf" command.

Discussion:

The objective of the given experiment is to approximate the concentration of dissolved oxygen in freshwater at 27 degree Celsius using Linear Interpolation with the given set of data. The given data were first stored in arrays to observe the change of concentration of oxygen with increasing temperature. A series of temperatures from 0 to 40 degree Celsius with an interval of 0.01 were taken to get a smooth curve. For each point, the code decides between which two known points, each point of the series exists. Identifying the points, the formula of linear interpolation is applied to get a piecewise function. As a result, a curve of Oxygen concentration vs temperature is generated.

This technique connects each point linearly to get a simple and accurate curve. The interpolated curve is blue in color and the plotted points are red. The Oxygen concentration at 27 degree Celsius is found using the 'find' function which is printed and shown as output. This value is quite close to the real value of oxygen concentration. This experiment shows the effectiveness of the function of linear interpolation in getting a simple and reliable curve using discrete data points.

(b)

Code:

```
clear all;
close all;
clc;
Temp = [0 8 16 24 32 40]; % forming array of the given Temperatures.
Ox_Con = [14.621 11.843 9.870 8.418 7.305 6.413]; % forming array of the given
concentrations of Oxygen(mg/L).
n = length(Temp); %Number of element in array of Temperature.

figure(1)
plot(Temp,Ox_Con,'r.','MarkerSize', 20);%Plotting the points in the graph.
grid on;% to get grid lines.
hold on;%to keep the plot unchanged.
Temp_s = 0:0.01:40;%Continuous value of Temperature from 0 to 40 with interval
of 0.01.
Temp_L = length(Temp_s);%Number of elements in Temp_s
Ox_Con_s = zeros(1,Temp_L); %array equal to the number of element in Temp_s
with zero values.
for k = 1:Temp_L % For every value of Temp_s
    P = ones (1,n); % Array from 1 to length of Temp to store every value of P.
    for i = 1:n %For every point of Temp.
        for j = 1:n %For every point of Temp.
            if j~=i %To exclude the point when j=i.
                P(i) = P(i)*(Temp_s(k)-Temp(j))/(Temp(i)-Temp(j));
            end
        end
        P(i) = P(i)* Ox_Con(i); % Each term of y.
    end
    Ox_Con_s(k) = sum(P); % Sum of all y points.
end

figure(1)
plot(Temp_s, Ox_Con_s,'b.');
```

% Plotting graph.

title("Lagrange's interpolating polynomial."); % Giving title of the graph.

xlabel('T (Celsius)'); % Labeling the x axis of graph.

ylabel('Oxygen (mg/L)'); %Labeling the y axis of graph.

tol = 0.0001;

x_f = find(abs(Temp_s-27.00)<=tol); %Using find function to get in which array
temp=27 exist.

Temp_val = Temp_s(x_f); % Temperature value stored from the array which contains
Temp_s =27.

y_f = Ox_Con_s(x_f); %At that specific array corresponding value of y.

fprintf("Estimated Oxygen concentration for %.2f degree Celsius is %.3f
mg/L\n",Temp_val, y_f); % Showing the output value.

Output:

```
Command Window  
New to MATLAB? See resources for Getting Started.  
Estimated Oxygen concentration for 27.00 degree Celsius is 7.968 mg/L  
>>
```

Figure 2(b)(i): Showing concentration oxygen at 27 degree Celsius using Lagrange Interpolating Polynomial.

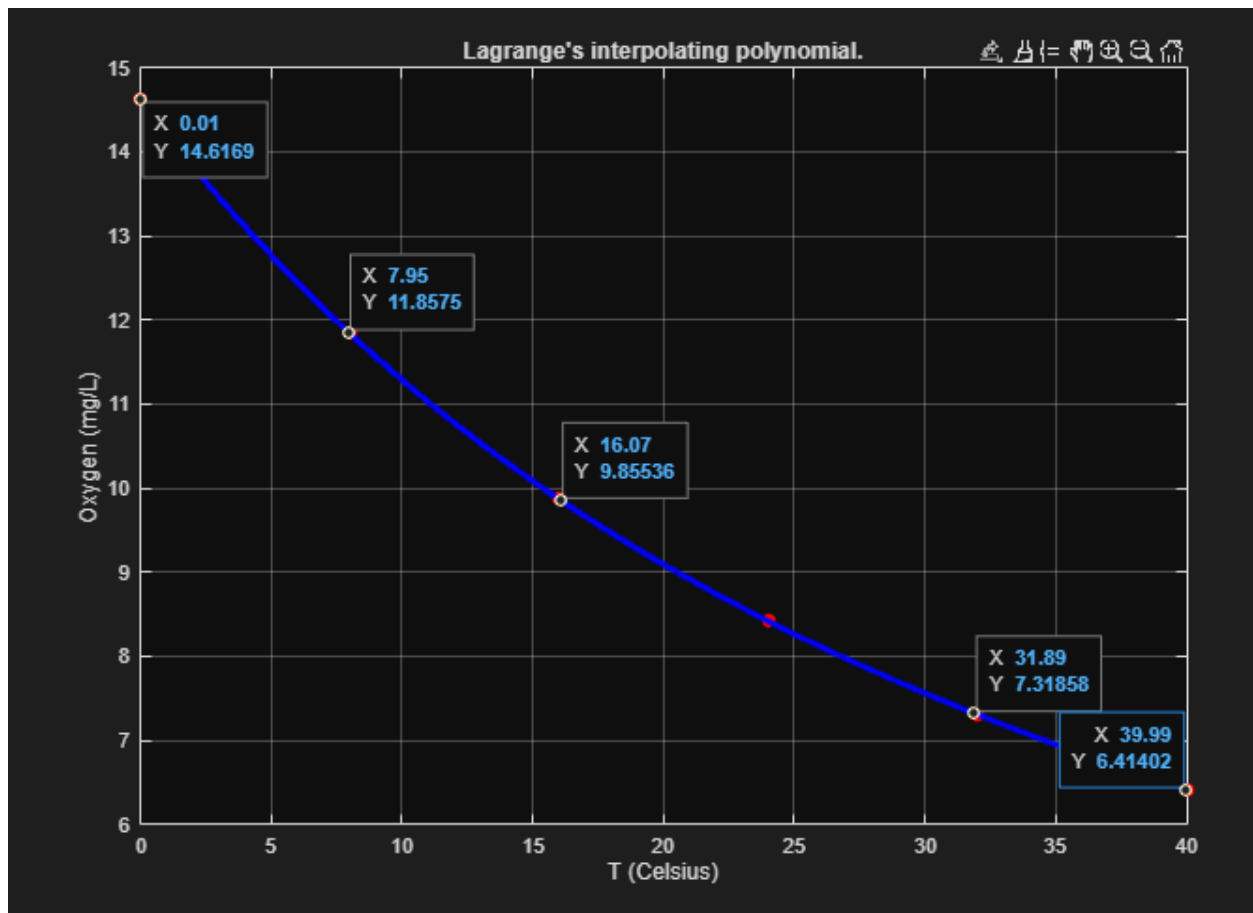


Figure 2(b)(ii): Graph showing points connected using Lagrange Interpolating Polynomial Method.

Explanation:

Array is formed with the values of given temperature and oxygen concentration. Using the “plot” command the points are plotted. In the “plot” command, ‘r’ and ‘MarkerSize’ is written to indicate the points to be red in color and size of those points is 20. ‘Grid’ and ‘hold on’ commands are used to get grid lines on the plot and to keep intact the plotted points in the graph even after the next plotted command is used respectively. A series of points of temperature is taken and an array is formed to store the corresponding values of y with the number of elements as in the series of points in temperature. “For” loop is run for every value of the series. The condition j not equal to i is given to avoid repeating of values. Each term of the polynomial is constructed by multiplying factors like $\frac{(x-x_j)}{(x_i-x_j)}$ where $j \neq i$. The loop runs for every given data point of a series of values of Temperature. Finally sums up all the values and plots it in the graph. To find the temperature at 27 degree Celsius, “find” command is used. If the difference of this point and the nearest point to this point is less or equal to 0.0001, that point is the value of temperature(27 degree Celsius) whose corresponding value is the required output (7.968 mg/L) and is shown using the “fprintf” function. The variable Temp_val is used to get the value of Temperature = 27 from the array.

Discussion:

The aim of this experiment was to apply Lagrange Interpolating Polynomial method to approximate the concentration of dissolved oxygen at 27 degree Celsius using the given discrete data. The given data were first plotted to visualize the points on the graph. A series of temperature values from 0 to 40 degree Celsius with an interval of 0.01 were taken to make a smooth curve. For each temperature, Lagrange basis polynomial was made by multiplying $\frac{T-T_j}{T_i-T_j}$ for every j not equal to i. All these values were multiplied by their corresponding oxygen concentration values and summed up. Then a smooth interpolated curve was formed with temperature along the x axis and Oxygen concentration along the y axis. The temperature closest to 27 degree Celsius was found using the ‘tolerance’ variable and ‘find’ function. The value of the concentration of oxygen was found to be 7.968 mg/L. Although the method has some limitations, the method is efficient for generating quite accurate curves.

(c)

Code:

```
close all;
clc;
Temp = [0 8 16 24 32 40]; % forming array of the given Temperatures.
Ox_Con= [14.621 11.843 9.870 8.418 7.305 6.413]; % forming array of the given
concentrations of Oxygen(mg/L).
n = length(Temp); %Number of element in array of Temperature.
Temp_s = 0:0.01:40; % Series is formed.
L = length(Temp_s); %Number of element in the series of Temp_s.
ys = zeros(1,L);%array of 0 elements are taken.
for i = 1:L % Loop works for every value of Temp_s.
    for j = 1: (n-1) %check between which point.
        if Temp_s(i)>= Temp(j) && Temp_s(i)<Temp(j+1)
            a =j; %lower index.
            b = j+1;%upper index.
        end
    end
    ys(i) =
Ox_Con(a)+((Ox_Con(b)-Ox_Con(a))./(Temp(b)-Temp(a))).*(Temp_s(i)-Temp(a));%Line
ar Interpolation formula.
end
x_f = find(abs(Temp_s == 27));% Locate the element.
y_f= ys(x_f);% Find value of oxygen concentration for which temperature is
27degree Celsius.
Exact_value = 7.986;
error_percent_Linear = ((abs(Exact_value - y_f))./ Exact_value).*100;
fprintf("For Linear Interpolation Error percentage is %.3f
percent\n",error_percent_Linear);
Ox_Con_s = zeros(1,L); %array equal to the number of element in Temp_s with
zero values.
for k = 1:L % For every value of Temp_s
    P = ones (1,n); % Array from 1 to length of Temp to store every value of P.
    for i = 1:n %For every point of Temp.
        for j = 1:n %For every point of Temp.
            if j~=i %To exclude the point when j=i.
                P(i) = P(i)*(Temp_s(k)-Temp(j))/(Temp(i)-Temp(j));
            end
        end
        P(i) = P(i)* Ox_Con(i); % Each term of y.
    end
    Ox_Con_s(k) = sum(P); % Sum of all y points.
end
tol = 0.0001;
```

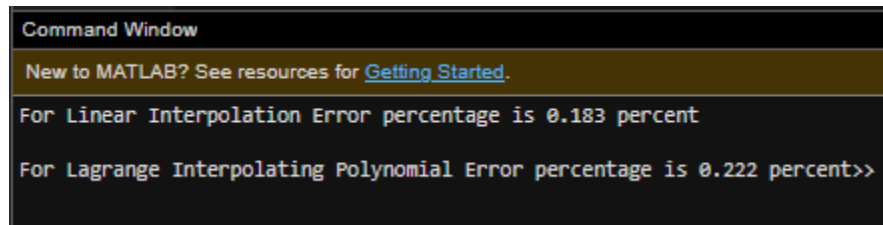


```

x_f = find(abs(Temp_s-27.00)<=tol);%Using find function to get in which array
temp=27 exist.
Temp_val = Temp_s(x_f);% Temperature value stored from the array which contains
Temp_s =27.
y_f = Ox_Con_s(x_f);%At that specific array corresponding value of y.
error_percent_Lagrange = ((abs(Exact_value - y_f))./Exact_value).* 100;
fprintf("\nFor Lagrange Interpolating Polynomial Error percentage is %.3f
percent",error_percent_Lagrange);

```

Output:



```

Command Window
New to MATLAB? See resources for Getting Started.
For Linear Interpolation Error percentage is 0.183 percent
For Lagrange Interpolating Polynomial Error percentage is 0.222 percent>>

```

Figure 2(c): Showing Percentage of Error found using Linear Interpolation and Lagrange Interpolating Polynomial.

Explanation:

Initially, two arrays are formed with the given values of temperature and Oxygen concentration. Then, a series of temperature values is taken with an interval of 0.01 and another array (y_s) is taken to store the corresponding value of this series. After that loop is run to apply the Linear Interpolation formula. From this loop, the value of oxygen concentration at 27 degree Celsius is found out by using the 'find' function. In order to find error percentage, this value of oxygen concentration is subtracted from the exact value of oxygen concentration (7.986 mg/L) and then divided by the exact value of oxygen concentration. The output is shown as a percentage which is 0.183%. Again, another loop is run to get the Lagrange Interpolation Polynomial. From this loop, the value of oxygen concentration at 27 degree Celsius is found out by using the 'find' function. After this, the percentage of error is found by subtracting the value of oxygen concentration at 27 degree Celsius from the exact value of oxygen concentration at 27 degree Celsius and dividing by the exact value. The output is shown as a percentage which is 0.222%.

Discussion:

The given experiment throws light on approximating the dissolved oxygen concentration at 27 degree Celsius by using Linear Interpolation as well as Lagrange Interpolating Polynomials. The given data of oxygen concentration and temperature were first stored in arrays and then a smooth temperature series from 0 to 40 degree Celsius was taken to generate a smooth interpolated curve. The Linear Interpolation formula computes the oxygen concentration using a straight line formula between two neighboring points which gives a piecewise linear approximation of the relationship between oxygen concentration and temperature.

On the contrary, an order 5 polynomial was generated using the set of values of temperature for Lagrange Interpolating polynomials. Each of the Lagrange basis polynomials add to the interpolated curve. As a result, a smooth approximation of the curve is observed.

After computing Linear Interpolation and Lagrange Interpolated Polynomials, the oxygen concentration at 27 degree Celsius was extracted and compared with the known values 7.986 mg/L. The linear interpolation gave a small error of 0.183% while Lagrange Interpolating Polynomial gave a higher error of 0.222% due to increased sensitivity, accuracy and oscillatory behaviour. Overall, the experiment demonstrates interpolation as a useful technique in estimating functions and to show that higher order polynomials give more accurate value.

(3)

Objective:

The objective of this experiment is to find eight equally spaced points of the function $f(t) = \sin^2(t)$ with the interval from 0 to 2π and to make a seventh order interpolating polynomial using MATLAB to estimate the function across the interval.

Theoretical Background:

Interpolation is a numerical technique used to find the approximate value of a function if only its discrete points are known. If $n+1$ data points are given, a n th order polynomial can be constructed that passes each of the points.

The function $f(t) = \sin^2(t)$ with 8 data points provides the sampled data for interpolation which ensures uniform sampling. For seventh order polynomial function Lagrange interpolation formula is

$$P(t) = \sum_{i=0}^7 f(t_i) L_i(t)$$

Here, each lagrange basis polynomial is

$$L_i(t) = \prod_{\substack{j=0 \\ j \neq i}}^7 \frac{t - t_j}{t_i - t_j}$$

This polynomial passes through all the functions and approximates the required points.

The interpolating polynomial helps us to make a smooth function with the given discrete values.

Problem Statement:

Problem-3: Generate eight equally-spaced points from the function using MATLAB:

$$f(t) = \sin^2(t)$$

from $t = 0$ to 2π . Fit these data with a seventh-order interpolating polynomial.

Code:

```
clear all;
close all;
clc;

t = 0:2.*(pi./7):2.*pi; %Eight equally spaced point.
t_Length = length(t); %number of element in t.
fprintf("Eight equally spaced points are\n");
f = sin (t).* sin(t);
fprintf("%f ", f);% Showing the value of eight equally spaced points.
figure(1)
plot(t,f,'b.','MarkerSize',20);
hold on;

t_s = 0 : 0.001: 2.*pi; % continous value from 0 to 2pi with interval of 0.01.
L = length(t_s);%number of elements in t_s.
f_s = zeros(1,L);%array with number of element for corresponding value of t_s.

for k = 1:L %for all value of t_s.
    p = ones(1,t_Length);%array with number of element as in t_s.
    for i= 1:t_Length % for every value of t_s
        for j = 1:t_Length % for every value of t_s
            if j~=i % to exclude one of the value where j=i.
                p(i) = p(i) * ((t_s(k)-t(j))/((t(i) - t(j))));
            end
        end
        p(i) = p(i)*f(i);% Each term of f_s.
    end
    f_s(k) = sum(p);% Sum of all f_s points.
end

figure(1)
plot(t_s,f_s,'r.');// Plotting graph.
title('Seventh-order Interpolating Polynomial of sin2(t)');% Giving title of the graph.
```

```
xlabel('t');% Labeling the x axis of graph.  
ylabel('f(t)');% Labeling the y axis of graph.  
grid on;% to get grid on.
```

Output:

```
Command Window  
New to MATLAB? See resources for Getting Started.  
Eight equally spaced points are  
fx 0.000000 0.611260 0.950484 0.188255 0.188255 0.950484 0.611260 0.000000 >>
```

Figure 3(i): Eight equally spaced points are shown.

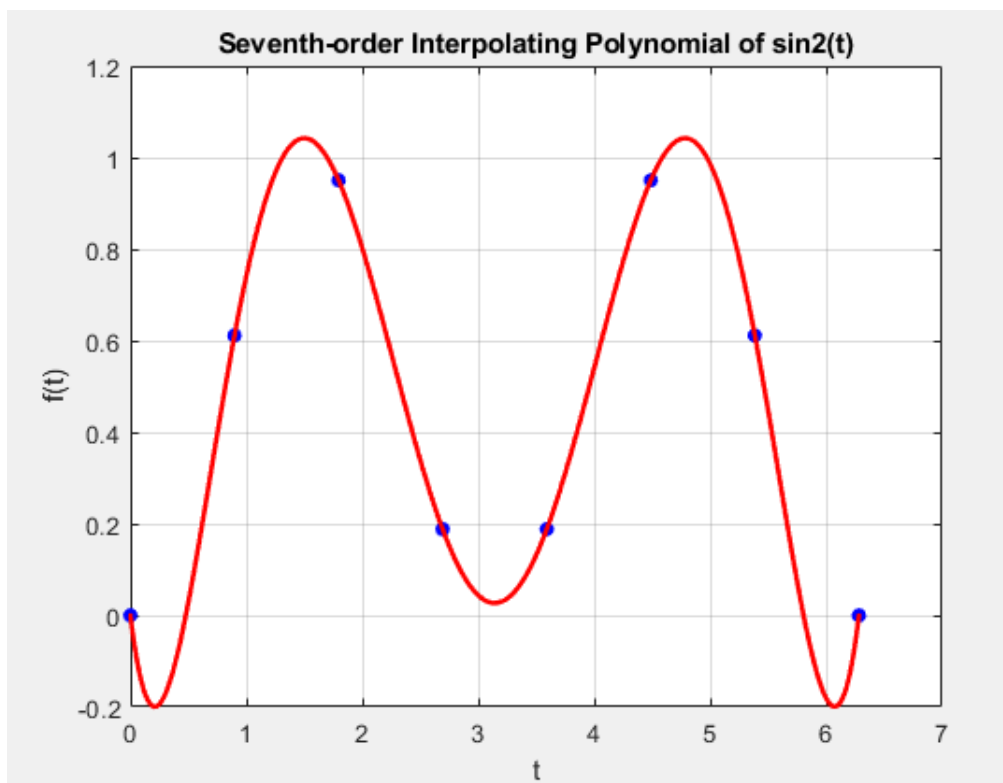


Figure 3(ii): Seventh Order Lagrange Interpolating Polynomial is shown.

Explanation:

At first, eight equally spaced points from 0 to 2π are made by dividing 2π by 7 and the points are used in the $\sin^2(t)$ to get the output in Figure 3(i). The points are plotted in the figure (blue colored) and by using 'Markersize' command, the size of the points is fixed. 'hold on' is used to keep these points intact when the next plot command is used. Then a continuous value of 't' is taken and an array (f_s) is made to store its corresponding values. A loop is run for every value of the series of values of 't'. Then an array is made (p) to store every term of $p(i)$. Finally all the values are summed up and plotted in the graph in a red colored line. 'title', 'xlabel', 'ylabel' and 'grid on' commands are used to write the title of the graph, labeling the x axis, labeling the y axis and to get grid lines on the graph respectively.

Discussion:

In the experiment, the first goal was to get eight equally spaced points and approximate the function $f(t) = \sin^2(t)$ over the interval 0 to 2π . Eight equally spaced points were made by dividing the interval by 7. These points served as a data for making 7th order Lagrange Interpolating Polynomials. This method was selected as it ensures more correctness. MATLAB was used to form these interpolations manually to allow us to see each basis function contribution to the approximation of the curve.

The MATLAB program computed the polynomial value in dense grid lines and ensured a smooth approximation of the given function. The plot in the graph shows the discrete data points as well as the continuous interpolating polynomial. The interpolated curve follows the shape of the sine function closely. Overall, this experiment shows how polynomial interpolation can generate a smooth function from discrete values and shows accuracy in scientific calculations.